Universidade de Aveiro

Departamento de Electrónica, Telecomunicações e Informática

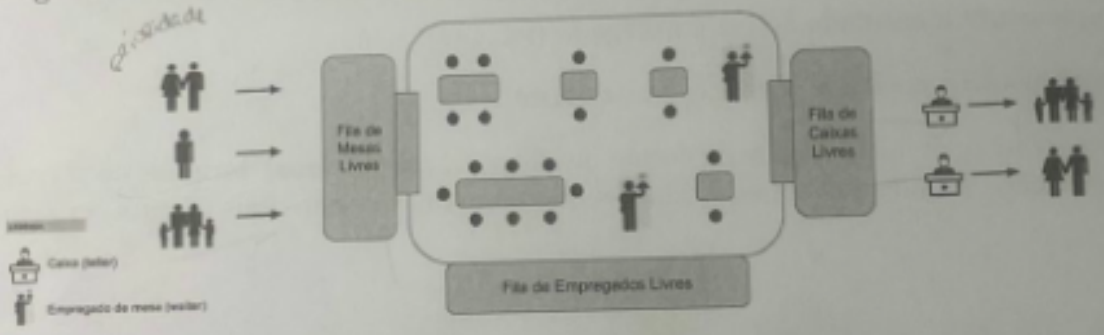
Sistemas de Operação / Fundamentos de Sistemas Operativos

(Ano lectivo de 2024/2025) 8 de Novembro de 2024

Exame prático NM

Nome: _____ NMec: _____

Pretende-se simular o funcionamento de um restaurante com clientes (*client*), empregados para servir comida (*waiter*) e caixas para pagamento (*teller*). Cada cliente tem um nome (de família) e é composto por um determinado número de pessoas. Um cliente, quando chega ao restaurante, ocupa uma mesa livre, come, paga e vai-se embora. A mesa a ocupar tem de ter um número de lugares suficientes para essa lotação (a implementação sequencial fornecida já resolve este problema). A gestão das mesas livres é feita com a fila ordenada `free_tables`, usando o "truque" de ordenar crescentemente os elementos da lista pela lotação das mesas; assim, na procura de uma mesa livre com uma locação mínima, até que exista, é escolhida uma mesa livre com a lotação mais próxima (todo o código de selecção de mesas já está feito pelo que não precisam de se preocupar com isso, nem é necessário compreender como está implementado)[1]. A gestão da escolha do empregado e do caixa é feita recorrendo, respectivamente, às filas `free_waiters` e `free_tellers`.
A figura seguinte ilustra a estrutura do restaurante:



O código fornecido implementa todos os aspectos sequenciais da simulação, tendo havido o cuidado (tal como no exercício de treino) de indicar por comentários as zonas do código que, com grande probabilidade, precisam de ser alteradas (acrescentando o código concorrente, de lançamento de entidades activas, comunicação e sincronização entre elas e de gestão dos recursos de suporte à concorrência). Zonas do código sem esses comentários, em princípio, não necessitam de ser mexidas. Desenvolva uma versão concorrente deste simulador (com threads/mutex/variáveis-de-condição [e, eventualmente, semáforos] ou com processos/memória-partilhada/semáforos).
Tenha também em atenção os seguintes aspectos:

- É fornecida a implementação (sequencialmente correcta) de um módulo para filas ordenadas (implementado com um array circular). No uso de filas sem ordenação (para solicitar empregados e caixas livres) este módulo é utilizado com uma ordenação fixa (valor 1);
- Como indicado atrás, o ciclo de vida de um cliente consiste simplesmente em entrar no restaurante e escolher mesa (fila), solicitar um empregado (fila), esperar que o empregado traga a comida, comer, libertar a mesa (fila), solicitar um caixa (fila), esperar pela confirmação do pagamento pelo caixa e terminar (ver `client_life`).
- O ciclo de vida de cada empregado (*waiter*) consiste em estar à espera de ser solicitado por um cliente (gerido pela fila `free_waiters`), comunicar com o cliente indicando que este já tem a refeição, e voltar ao início (até que a simulação termine) (ver `waiter_life`).

[1] Está garantida a existência de pelo menos uma mesa com capacidade para o número máximo de pessoas por cliente.

- O ciclo de vida de cada caixa (*teller*) consiste em estar à espera de ser solicitado por um cliente (gerido pela fila `free_tellers`), registar no cliente que este já tem a refeição paga, e voltar ao início (até que a simulação termine) (ver `teller_life`).
- O código fornecido contém a biblioteca de concorrência fornecida durante as aulas. Pode ver a documentação executando um browser sobre o ficheiro `doc/index.html` (ex: `firefox doc/index.html&`). Esta biblioteca resolve o problema da programação defensiva na utilização das bibliotecas nativas de threads, mutexes, variáveis de condição, processos, memória partilhada e semáforos.
- O código fornecido faz uso de asserções, expressas com as macros `require`, `ensure` e `check` que indicam as condições que se têm de verificar num código correcto.

Sugere-se que implemente esta simulação seguindo os seguintes passos:

1. Altere o módulo da fila ordenada (`ordered_queue`) por forma a que passe a ser um módulo partilhado seguro. Nesse sentido, tenha em atenção que algumas asserções passam a ser pontos de sincronização condicional (a regra geral é muito simples: isso tem de acontecer sempre que a condição possa ser garantida por outra entidade activa)
2. Implemente a entidade activa cliente (`client`).
3. Implemente a entidade activa empregado (`waiter`).
4. Implemente a entidade activa caixa (`teller`).
5. Faça com quer a operação de fechar as filas (`close_oqueue`) termine de forma segura a simulação. As filas devem ser fechadas quando não existirem mais clientes no restaurante.